

DLDCALab 2: Assembly

Week 3

August 26, 2025

Introduction

In this week's lab, you shall be learning about how structs are placed in memory. You will also use xmm registers and few special instructions to deal with doubles (64 bit floats). The cheatsheet for assembly instructions can be found in section 1. Refer to section 2 for an overview on instruction to deal with doubles using xmm registers.

Necessary tools

We shall be using nasm in order to compile x86 assembly code.

Before starting the lab, ensure that the tool is installed in your system. This can be verified by running the following:

```
$ nasm
nasm: fatal: no input file specified
Type nasm -h for help.
```

Structure of submission/templates

```
your-roll-no/
|- task1/
    |- print_cmplx.asm (TODO)
    |- compile.sh
|- task2/
```

```

    |- basic_cmplx_arith.asm (TODO)
    |- compile.sh
|- task3/
    |- adv_cmplx_arith.asm (TODO)
    |- compile.sh

```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of your-roll-no.tar.gz. The command given below can be used to do the same

```
$ tar -cvzf your-roll-no.tar.gz your-roll-no/
```

Tasks

An overview of tasks can be seen in TODO.md. **Please go through this file before starting the lab.** This task consists of two subtasks. This task is expected to be submitted in-lab. The code for this task can be found inside the task1 folder.

Understanding structs in x86

Structs can be thought of as custom data structures placed continuously in memory. A struct has ‘fields’, where each field is a part of the struct. For example in this assignment complex number are represented as a struct with three fields. The first field is the name of the complex number which is a single byte character, the second field is a 64 bit double to represent the real part of the complex number and the third field is the imaginary part of the complex number.

Thus a complex number $3 + 4i$ with the name ‘a’ is represented as

```

complex1 :
    complex1_name db 'a'
    complex1_real dq 1.0
    complex1_img  dq 2.5

```

where dq represents a quad word (64 bits). But if you observe in the template code given the complex struct is represented as follows.

```

complex1 :
    complex1_name db 'a'
    complex1_pad  db 7 dup(0)
    complex1_real dq 1.0
    complex1_img  dq 2.5

```

To understand why this is the case we need to understand structure padding. Click [Here](#) to watch the video for the same. This video is also captioned and provided to you.

The usecase for structs is visible from how we use it to represent complex numbers.

Now, a big issue is how to access the fields of the struct in ASM. Well, as we discussed, the structure is just a continuous data structure. So, the struct's placement in memory can be visualized like:

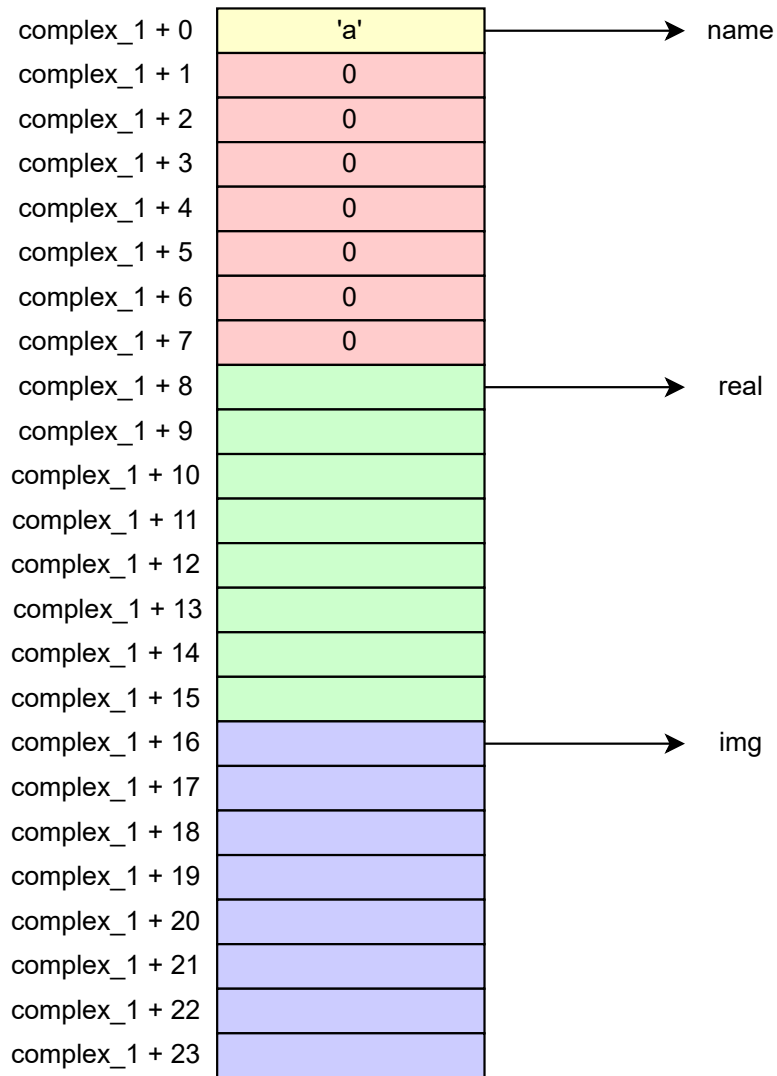


Figure 1: Layout of complex struct in memory: complex_1 is the address of the label complex_1

From the diagram it is clear to see that the individual fields can be accessed by offsetting the address of the complex struct by appropriate amounts (complex + 8) in the case of the field 'real'.

Few notes while using doubles with XMM registers

- To load a constant there is no immediate values supported, a simple workaround is to initialize constants in the data segment and use them from memory
- When initializing doubles make sure to include a decimal point (Eg. 6.0 instead of 6). This makes sure that the value is interpreted as a double and not as an int.
- To pass double arguments to functions use xmm0, xmm1, xmm2... in that order. Further xmm registers are caller saved and need to be saved if they are needed in the future by the caller. In this case push xmm does not work.
- Instead of push, manually offset the stack pointer and store in the stack to push. Similarly instead of pop load from stack and increment the stack pointer manually.¹

```
sub rsp, 16      ; push
movaps [rsp], xmm*
;function call here
movaps xmm*, [rsp] ; pop
add rsp, 16
```

Replace xmm* with the relevant xmm register

Task 1: Printing complex numbers

In this task you will be writing a custom function that takes in the address of a complex number (say a) and a message string (say str) and prints the following.

```
str => Real: a.real Imaginary: a.img
```

This can be easily achieved using printf and format string. It is as simple as

```
printf(" %s => Real: %f Imaginary: %f ", str, a.real, a.img);
```

¹Note that the xmm register is 128 bits in size, the instructions we use only take 64 bits from it as a float value

But this begs the question how to we use such functions that we took for granted in C/C++. These are standard library functions which use syscalls under the hood to do complex tasks, in this case to print values to the screen. The implementations of these functions can be used in raw assembly code by just referring to them²

`printf` can simply be referred to and the implementation of the underlying standard library is used. We do not have to implement it.

To show how this is done we have implemented a function `print_float` that takes a string say `str` and the address of a float say `a` and makes a call to `printf` equivalent to

```
printf("%s => Float: %f",str,a);
```

Refer to the implementation of the same a create a equivalent `print_cmplx` to complete this task. Note that your implementation takes a string apart from the complex number input. The format string to be used is given to you in the `print_complex.asm` file. The expected output for this task is as follows:

```
Demo of print_float => Float: 1.000000
Testing print complex: => Real: 1.000000 Imaginary: 2.500000
```

In this example "Testing print complex" is also taken as an argument by the function. The format string for the same

```
"%s => Real: %f Imaginary: %f"
```

is given in the file as a value in the `.data` section.

Task 2: Complex numbers arithmetic (Basic)

Addition with complex numbers:

Our complex number struct can be represented as follows:

```
struct complex{
    char name;
    double real;
    double img;
};
```

²Additional note of interest: To access the implementations of these functions we need to link the code with `gcc` instead of the direct link, info for those interested :)

The next subtask in the inlab submission is to implement the addition of two complex numbers in assembly.

The addition function you implement must take memory addresses as its parameters:

1. Source address of complex 1
2. Source address of complex 2
3. Destination address of result

In C, this could become something along the lines of:

```
void add_cmplx(struct complex* a, struct complex* b, struct complex* c){
    c->real = a->real + b->real
    c->img = a->img + b->img
}
```

Task 3: Complex numbers arithmetic (advanced)

Reciprocal with complex numbers:

Our complex number struct can be represented as follows:

```
struct complex{
    char name;
    double real;
    double img;
};
```

The next subtask in the inlab submission is to implement the taking the reciprocal of a complex number.

The reciprocal function you implement must take memory addresses as its parameters:

1. Source address of complex
2. Destination address of result

In C, this could become something along the lines of:

```
void recip_cmplx(struct complex* a, struct complex* b){
    b->real = a->real / (a->real^2 + a->img^2);
    b->img = - a->img / (a->real^2 + a->img^2);
}
```

}

Fill in the templates given to finish these tasks. **Note** that for tasks 2,3 the `print_cmplx` implemented in task 1 is used. The compile scripts automatically take care of this assuming you have the right directory structure. Thus to verify correct outputs of tasks 2,3 one needs to finish task 1. To run your code, you may run (from the corresponding task directory):

```
$ bash compile.sh
$ ./print_cmplx
Demo of print_float => Float: 1.000000
Testing print complex: => Real: 1.000000 Imaginary: 2.500000
```

```
$ bash compile.sh
$ ./basic_cmplx_arith
Testing Addition => Real: 4.500000 Imaginary: 6.500000
```

```
$ bash compile.sh
$ ./adv_cmplx_arith
Testing Reciprocal => Real: 0.137931 Imaginary: -0.344828
```


1 Assembly instructions cheatsheet (non-exhaustive)

Instruction	Operands	Effect
mov	reg, reg/*mem/imm	$\text{reg} \leftarrow \text{reg} / * \text{mem} / \text{imm}$
mov	*mem, reg	$* \text{mem} \leftarrow \text{reg}$
add	reg, reg/*mem/imm	$\text{reg} \leftarrow \text{reg} + \text{reg} / * \text{mem} / \text{imm}$
add	*mem, reg/imm	$* \text{mem} \leftarrow * \text{mem} + \text{reg} / \text{imm}$
sub	reg, reg/*mem/imm	$\text{reg} \leftarrow \text{reg} - \text{reg} / * \text{mem} / \text{imm}$
sub	*mem, reg/imm	$* \text{mem} \leftarrow * \text{mem} - \text{reg} / \text{imm}$
xor	reg, reg/*mem/imm	$\text{reg} \leftarrow \text{reg} \oplus \text{reg} / * \text{mem} / \text{imm}$
xor	*mem, reg/imm	$* \text{mem} \leftarrow * \text{mem} \oplus \text{reg} / \text{imm}$
and	reg, reg/*mem/imm	$\text{reg} \leftarrow \text{reg} \wedge \text{reg} / * \text{mem} / \text{imm}$
and	*mem, reg/imm	$* \text{mem} \leftarrow * \text{mem} \wedge \text{reg} / \text{imm}$
or	reg, reg/*mem/imm	$\text{reg} \leftarrow \text{reg} \vee \text{reg} / * \text{mem} / \text{imm}$
or	*mem, reg/imm	$* \text{mem} \leftarrow * \text{mem} \vee \text{reg} / \text{imm}$
imul	reg, reg/*mem/imm	$\text{reg} \leftarrow \text{reg} \times \text{reg} / * \text{mem} / \text{imm}$
imul	*mem, reg/imm	$* \text{mem} \leftarrow * \text{mem} \times \text{reg} / \text{imm}$
div	reg	$\text{rax} \leftarrow (\text{rdx}:\text{rax}) / \text{reg}$ $\text{rdx} \leftarrow (\text{rdx}:\text{rax}) \% \text{reg}$
cmp	reg/*mem, reg/imm	Sets flags based on subtraction: $\text{reg} / * \text{mem} - \text{reg} / \text{imm}$. ZF = 1 if equal, SF = 1 if result negative, CF = 1 if borrow (unsigned), OF = 1 if signed overflow.
test	reg/*mem, reg/imm	Sets flags based on bitwise AND: $\text{reg} / * \text{mem} \wedge \text{reg} / \text{imm}$. ZF = 1 if result is zero, SF = 1 if high bit set, CF and OF are cleared. No result is stored.
inc	reg/*mem	$\text{reg} / * \text{mem} \leftarrow \text{reg} / * \text{mem} + 1$
dec	reg/*mem	$\text{reg} / * \text{mem} \leftarrow \text{reg} / * \text{mem} - 1$
jmp	label	Unconditional jump to label
je / jz	label	Jump if Zero Flag (ZF) = 1
jne / jnz	label	Jump if Zero Flag (ZF) = 0
jl	label	Jump if Less (SF $\neq$ OF)
jle	label	Jump if Less or Equal (ZF = 1 or SF $\neq$ OF)
jg	label	Jump if Greater (ZF = 0 and SF = OF)
jge	label	Jump if Greater or Equal (SF = OF)
syscall		Performs a syscall with syscall number stored in rax Arguments passed in other registers starting from rdi
push	reg	Pushes the value in reg onto the stack
pop	reg	Pops the value on top of the stack onto reg
call	label	Calls the function called label

Table 1: Common x86-64 Instructions and Their Effects

2 xmm register instructions

Instruction	Operands	Effect
movsd	xmm, xmm/*mem	$\text{xmm} \leftarrow \text{xmm}/*\text{mem}$
movsd	*mem, xmm	$*\text{mem} \leftarrow \text{xmm}$
addsd	xmm, xmm/*mem	$\text{xmm} \leftarrow \text{xmm} + \text{xmm}/*\text{mem}$
subsd	xmm, xmm/*mem	$\text{xmm} \leftarrow \text{xmm} - \text{xmm}/*\text{mem}$
mulsd	xmm, xmm/*mem	$\text{xmm} \leftarrow \text{xmm} \times \text{xmm}/*\text{mem}$
divsd	xmm, xmm/*mem	$\text{xmm} \leftarrow \text{xmm} \div \text{xmm}/*\text{mem}$
movq	xmm, reg/*mem	$\text{xmm} \leftarrow \text{64-bit reg}/*\text{mem}$
movq	reg/*mem, xmm	$\text{64-bit reg}/*\text{mem} \leftarrow \text{xmm}$
movaps	xmm, xmm/m128	$\text{xmm} \leftarrow \text{128-bit xmm/mem (aligned)}$
movaps	xmm/m128, xmm	$\text{128-bit xmm/mem (aligned)} \leftarrow \text{xmm}$

Table 2: Common XMM Double-Precision Instructions